

1-)Quality Attributes

1. Simplicity

Since we are a junior development team, avoiding over-engineering is one of our main priorities.

- Why:

Our system includes AI analysis, plagiarism detection, sandbox execution, and LMS integration. If we design an overly complex architecture (e.g., microservices from the beginning), it may increase development difficulty and reduce maintainability.

- Implementation:

We separate the system into clear layers:

Presentation Layer (Web Interface + Moodle Integration)

Business Logic Layer (Grading, AI Analysis, Plagiarism Detection)

Data & Infrastructure Layer (Database, Docker Sandbox, Cloud Services)

This separation ensures readable, maintainable, and debuggable code.

2. Agility (via Modularity)

To allow fast development and easy feature updates, we focus on modular design within the Business Logic Layer.

- Why:

Assessly contains independent components such as:

Automated Test Engine

AI-Based Code Analysis

Plagiarism Detection

Smart Grading Module

If one module fails (e.g., AI detection), the core grading system should still function properly.

- Implementation:

Each major feature is implemented as an independent module. This allows:

Parallel development by team members

Easier debugging

Faster feature updates

Reduced system-wide risks

3. Scalability (Scalability-Ready Design)

The system must handle growing numbers of users and submission loads, especially near deadlines.

- Why:

In large programming courses, hundreds of students may submit code simultaneously. Additionally, sandbox execution and AI analysis are computationally expensive operations.

- Implementation:

Docker-based isolated execution per submission

Stateless API-based backend

Scalable database design (cloud-ready)

Future-ready design to offload AI analysis to external cloud services

This allows horizontal scaling without architectural redesign.

4. Security

Security is critical since students submit executable code.

- Why:

Malicious or infinite-loop code could damage system stability or expose vulnerabilities.

Student data and academic records must also be protected.

- Implementation:

Docker sandbox isolation for each submission

Runtime and memory limits enforcement

Secure API authentication

Role-based access control (Instructor / Student / Admin)

Encrypted communication (HTTPS)

Security ensures safe execution and protects institutional infrastructure.

5. Reliability

The system must function consistently during peak usage periods.

- Why:

System crashes during submission deadlines would directly affect academic evaluation.

- Implementation:

Graceful error handling

Submission queue management

Logging and monitoring system

Database backup strategies

This ensures system stability and consistent grading results.

2-)Architecture Style

- Layered Architecture
- We chose Layered Architecture because it provides:
- Simplicity → Clear separation of responsibilities
- Modularity → Independent feature development
- Scalability → Expandable backend design
- Security → Controlled data and execution flow.

3-)High-Level Diagram

